

СОДЕРЖАНИЕ

Введение	4
Перечень выполненных работ	5
1. Инициализация проекта и базовая архитектура	5
2. Логика маршрутизации и навигации	7
3. Работа с данными: Поиск и Бронирование	9
4. Реализация мультиязычности интерфейса (i18n)	10
5. Разработка функционала «Избранное» с персистентностью данных.....	12
6. Написание модульных тестов для критических компонентов	16
Заключение.....	18

ВВЕДЕНИЕ

Современная индустрия информационных технологий характеризуется активным развитием веб-платформ, ориентированных на автоматизацию бизнес-процессов и повышение удобства взаимодействия с конечными пользователями. В условиях высокой конкуренции особое значение приобретает качество клиентских веб-приложений, их архитектурная устойчивость, безопасность пользовательских сценариев и производительность интерфейса. В этой связи прохождение технологической (проектно-технологической) практики является важным этапом подготовки будущих специалистов в области программной инженерии и веб-разработки.

Целью данной практики являлось получение практического опыта разработки клиентской части современного веб-приложения на базе фреймворка Next.js с использованием библиотеки React. В рамках практики осуществлялась работа над реальным проектом, включающим функционал маршрутизации, авторизации пользователей, управления глобальным состоянием, валидации пользовательских данных и интеграции вспомогательных сервисов.

В процессе выполнения задач практики были изучены и применены на практике ключевые аспекты промышленной веб-разработки: организация архитектуры фронтенд-приложения, работа с провайдерами состояния и контекста, настройка маршрутизации и защиты пользовательских сценариев, обработка параметров URL, а также обеспечение корректной инициализации приложения при серверном и клиентском рендеринге. Особое внимание уделялось вопросам масштабируемости кода, повторного использования компонентов и соответствия современным стандартам разработки.

Таким образом, практика была направлена не только на закрепление теоретических знаний, полученных в ходе обучения, но и на формирование целостного представления о процессе разработки веб-приложений в реальных условиях, максимально приближённых к требованиям индустрии.

ПЕРЕЧЕНЬ ВЫПОЛНЕННЫХ РАБОТ

1. Инициализация проекта и базовая архитектура

Дата выдачи: 6 октября 2025 г.

Автор задачи: руководитель практики.

Срок для исполнения: 3 рабочих дня.

Формулировка задачи:

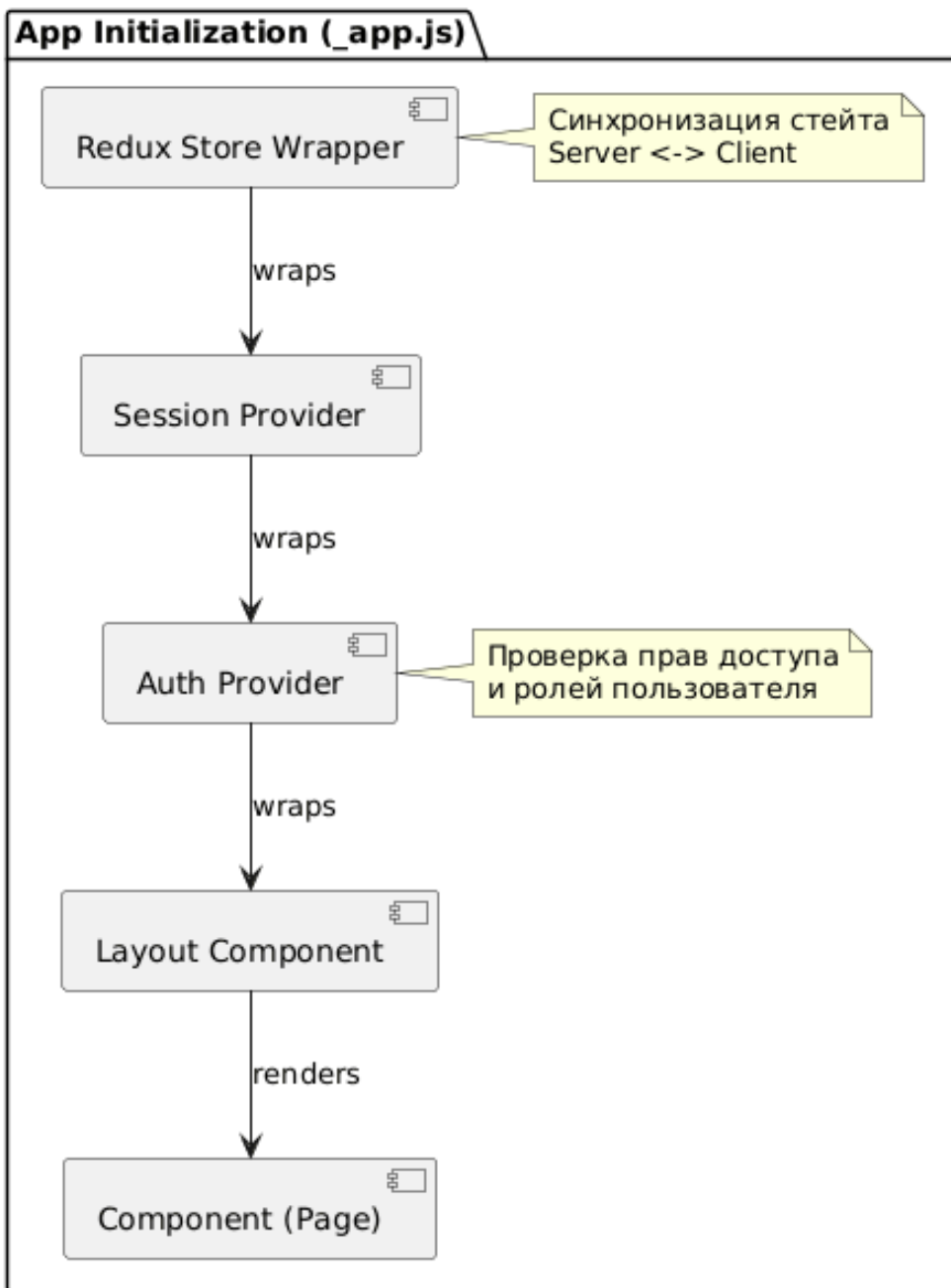
Для старта разработки веб-приложения требовалось развернуть масштабируемую архитектуру на базе фреймворка Next.js. Необходимо было настроить SEO-параметры главной страницы, интегрировать систему аутентификации и обеспечить глобальное управление состоянием (State Management) для бесшовной работы серверного и клиентского рендеринга.

Описание выполненных работ:

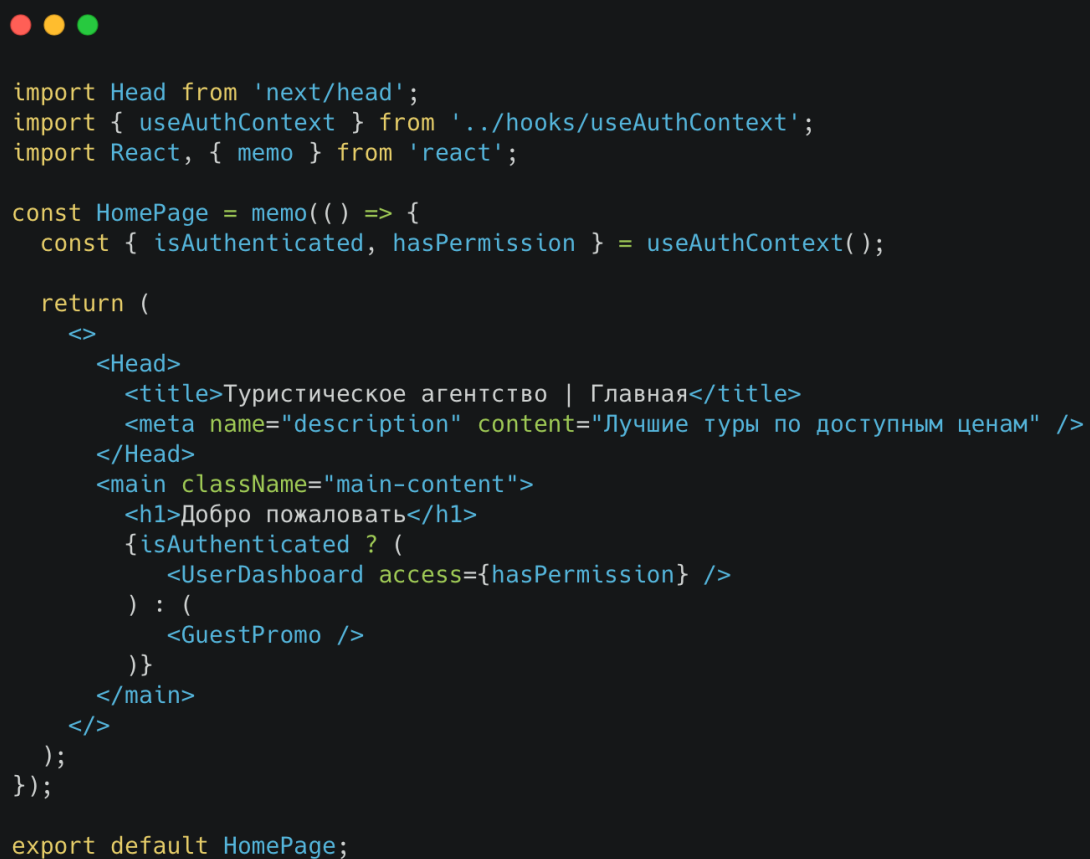
На начальном этапе была сформирована файловая структура проекта. Реализация главной страницы потребовала настройки метаданных (компонент Head) для SEO и интеграции с системой авторизации. Была внедрена архитектура, разделяющая слои представления (components), бизнес-логики (hooks/services) и утилит.

Для управления глобальным состоянием был модифицирован корневой файл `_app.js`. Внедрение Redux и провайдера сессий (NextAuth) потребовало создания обертки, которая гарантирует корректную гидратацию состояния при переходе с сервера на клиент.

В коде главной страницы использован кастомный хук для проверки прав доступа и условного рендеринга контента.



**Рисунок 1.1 - Архитектурная модель вложенности
провайдеров данных**



```

import Head from 'next/head';
import { useAuthContext } from '../hooks/useAuthContext';
import React, { memo } from 'react';

const HomePage = memo(() => {
  const { isAuthenticated, hasPermission } = useAuthContext();

  return (
    <>
      <Head>
        <title>Туристическое агентство | Главная</title>
        <meta name="description" content="Лучшие туры по доступным ценам" />
      </Head>
      <main className="main-content">
        <h1>Добро пожаловать</h1>
        {isAuthenticated ? (
          <UserDashboard access={hasPermission} />
        ) : (
          <GuestPromo />
        )}
      </main>
    </>
  );
});

export default HomePage;

```

Рисунок 1.2 - Фрагмент кода. Реализация главной страницы (HomePage)

Фактический срок реализации: 3 рабочих дня.

Комментарий принимающего лица:

«Архитектура приложения заложена грамотно. Правильная иерархия провайдеров в `_app.js` обеспечивает стабильный доступ к данным сессии и Redux-хранилищу из любого компонента. Структура проекта готова к масштабированию».

2. Логика маршрутизации и навигации

Дата выдачи: 13 октября 2025 г.

Автор задачи: руководитель практики.

Срок для исполнения: 4 рабочих дня.

Формулировка задачи:

Необходимо было обеспечить безопасность пользовательских сценариев и реализовать механизм динамического отображения страниц туров. Требовалось создать систему защиты маршрутов (Route Guard), предотвращающую доступ к страницам без необходимых параметров, а также настроить генерацию страниц каталога на основе данных из URL.

Описание выполненных работ:

Разработан механизм защиты маршрутов, базирующийся на хуке `useEffect`. Алгоритм проверяет наличие обязательных параметров в адресной строке и данных пользователя в `localStorage`. В случае несоответствия происходит автоматическая переадресация на безопасную страницу.

Использование динамического роутинга `Next.js` позволило реализовать генерацию страниц туров. Применен метод `getStaticPaths` совместно с `getStaticProps` и режимом `fallback: true`. Это решение обеспечивает мгновенную загрузку популярных туров (SSG) и генерацию «по требованию» для редко запрашиваемых, что положительно сказывается на производительности.



```
useEffect(() => {
  if (!router.isReady) return;

  const { tourId } = router.query;
  const storedUser = localStorage.getItem('user_data');

  setIsLoading(true);

  // Валидация обязательных параметров
  if (!tourId || !storedUser) {
    // Использование replace для исключения возврата на ошибочную страницу
    router.replace('/search?error=missing_params');
  } else {
    setIsLoading(false);
  }
}, [router.isReady, router.query]);
```

Рисунок 2.1 - Фрагмент кода 2. Логика валидации параметров (Route Guard)

Фактический срок реализации: 4 рабочих дня.

Комментарий принимающего лица:

«Логика маршрутизации реализована эффективно. Гибридный подход к рендерингу (SSG + Fallback) является оптимальным решением для каталога с большим количеством страниц. Механизм защиты маршрутов исключает возникновение ошибок при навигации».

3. Работа с данными: Поиск и Бронирование

Дата выдачи: 20 октября 2025 г.

Автор задачи: руководитель практики.

Срок для исполнения: 4 рабочих дня.

Формулировка задачи:

Ключевой бизнес-функцией системы является подбор и покупка туров. Требовалось разработать высокопроизводительный модуль фильтрации данных на клиенте, исключающий задержки интерфейса, а также реализовать форму бронирования со строгой валидацией вводимых данных (email, телефон, паспортные данные).

Описание выполненных работ:

Для фильтрации списка туров разработан кастомный хук `useTourFilter` с использованием мемоизации (`useMemo`). Это предотвращает лишние пересчеты массива данных при рендере других компонентов и обеспечивает плавность интерфейса.

Форма бронирования реализована с использованием библиотеки `react-hook-form`. Это решение позволило внедрить сложную валидацию (RegEx для email, проверка длины и формата телефона) без потери производительности, так как библиотека минимизирует количество перерисовок компонента (`Uncontrolled Components`) при вводе данных пользователем.

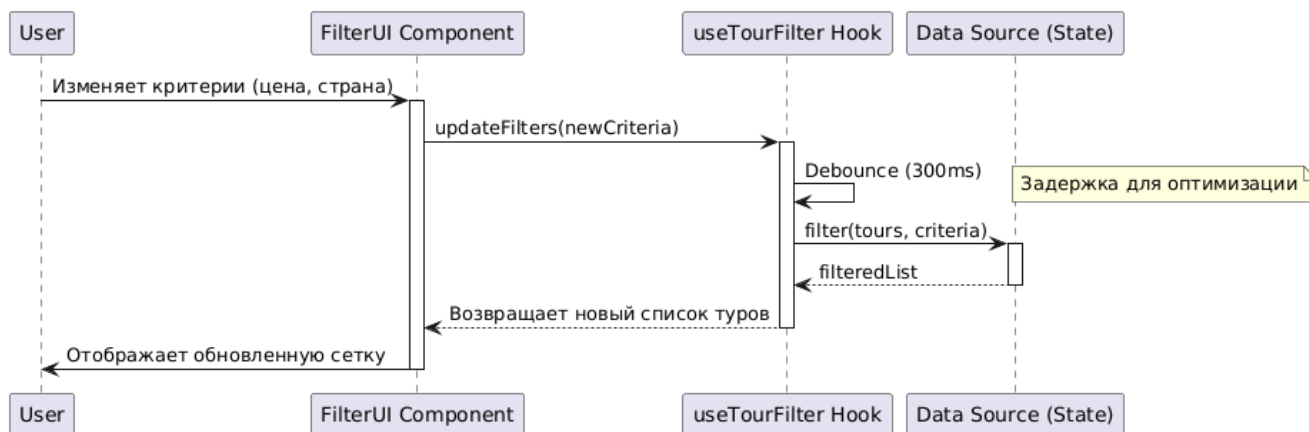


Рисунок 3.1 - Логика работы модуля фильтрации данных

Фактический срок реализации: 4 рабочих дня.

Комментарий принимающего лица:

«Функциональный блок реализован на высоком уровне. Оптимизация фильтрации через мемоизацию и использование специализированной библиотеки для форм демонстрирует глубокое понимание принципов производительности React-приложений».

4. Реализация мультиязычности интерфейса (i18n)

Дата выдачи: 27 октября 2025 г.

Автор задачи: руководитель практики.

Срок для исполнения: 4 рабочих дня.

Формулировка задачи:

Учитывая специфику туристического бизнеса, веб-приложение должно обслуживать международную аудиторию. Требовалось внедрить подсистему интернационализации для поддержки русского и английского языков. Необходимо было обеспечить автоматическое определение языка браузера пользователя, динамическую смену локали без перезагрузки страницы и разделение текстового контента от программного кода.

Описание выполненных работ:

Для решения задачи была выбрана библиотека `next-i18next`, обеспечивающая интеграцию стандартного `i18n`-решения с механизмом серверного рендеринга `Next.js`.

Текстовые константы были вынесены из компонентов в JSON-файлы переводов (`locales`), структурированные по папкам (`/public/locales/ru`, `/public/locales/en`).

В конфигурационном файле `next.config.js` настроены параметры маршрутизации: дефолтная локаль и список поддерживаемых языков. Это позволило автоматически генерировать префиксы в URL (например, `/en/tours`).

Разработан компонент-переключатель языка (`LanguageSwitcher`), который изменяет путь маршрута через `router.push`, сохраняя текущие параметры запроса. Использован хук `useTranslation` для динамической подстановки текстовых значений в интерфейсе.

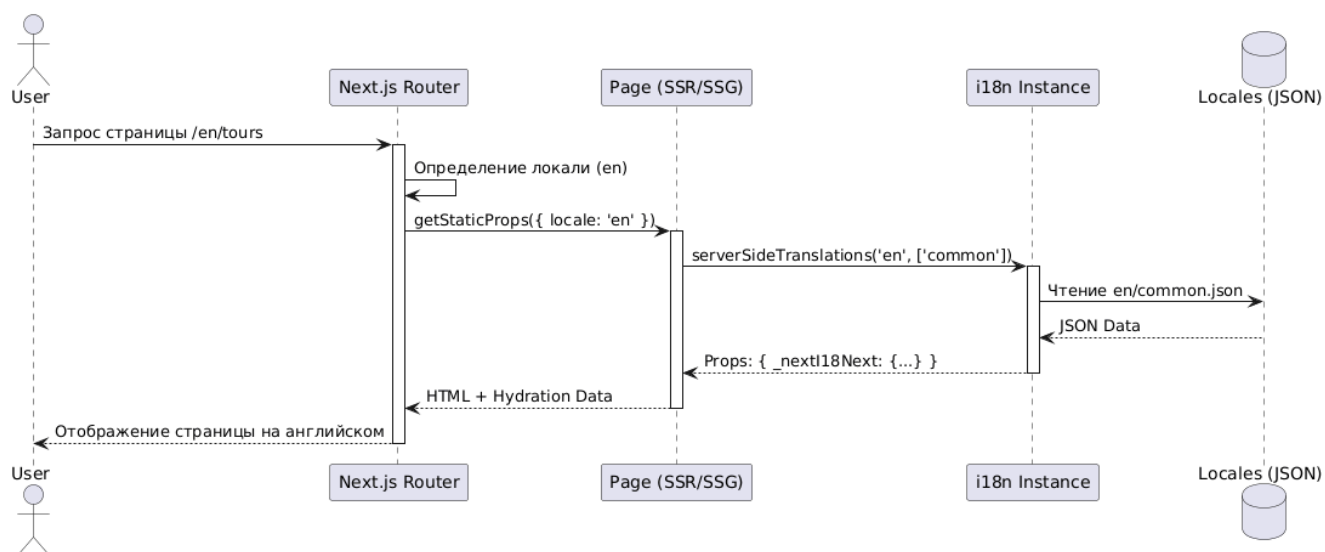


Рисунок 4.1 - Схема процесса загрузки локализованного контента

```

// pages/index.js
import { serverSideTranslations } from 'next-i18next/serverSideTranslations';
import { useTranslation } from 'next-i18next';

export const getStaticProps = async ({ locale }) => ({
  props: {
    // Загрузка переводов на сервере для конкретной локали
    ...(await serverSideTranslations(locale, ['common', 'home'])),
  },
});

const HomePage = () => {
  // Хук для получения функции перевода t
  const { t } = useTranslation('home');

  return (
    <section className="hero">
      <h1>{t('welcome_title')}</h1>
      <p>{t('welcome_subtitle')}</p>
      <button>{t('cta_button')}</button>
    </section>
  );
};

```

Рисунок 4.2 - Фрагмент кода. Реализация подключения переводов на странице

Фактический срок реализации: 4 рабочих дня.

Комментарий принимающего лица:

«Архитектура мультиязычности выстроена корректно. Вынос текстов в отдельные файлы значительно упростит дальнейшую поддержку и работу контент-менеджеров. Маршрутизация с языковыми префиксами работает стабильно и соответствует требованиям SEO».

5. Разработка функционала «Избранное» с персистентностью данных

Дата выдачи: 3 ноября 2025 г.

Автор задачи: руководитель практики.

Срок для исполнения: 3 рабочих дня.

Формулировка задачи:

Для повышения удержания пользователей (Retention Rate) требовалось реализовать возможность добавления туров в список «Избранное». Ключевым требованием являлось сохранение выбранных туров после закрытия вкладки браузера даже для неавторизованных пользователей, а также мгновенная визуальная реакция интерфейса при взаимодействии.

Описание выполненных работ:

В Redux-хранилище был создан новый слайс favoritesSlice, отвечающий за добавление, удаление и проверку наличия тура в списке.

Для обеспечения сохранности данных (персистентности) реализована синхронизация состояния Redux с localStorage браузера через middleware (промежуточное ПО) и хук useEffect. При инициализации приложения происходит гидратация (восстановление) списка из локального хранилища.

В UI-компонентах карточки тура добавлена кнопка-индикатор в виде сердца. Реализована анимация нажатия и оптимистичное обновление интерфейса (Optimistic UI) - изменение состояния происходит мгновенно, не дожидаясь завершения фоновых операций записи.

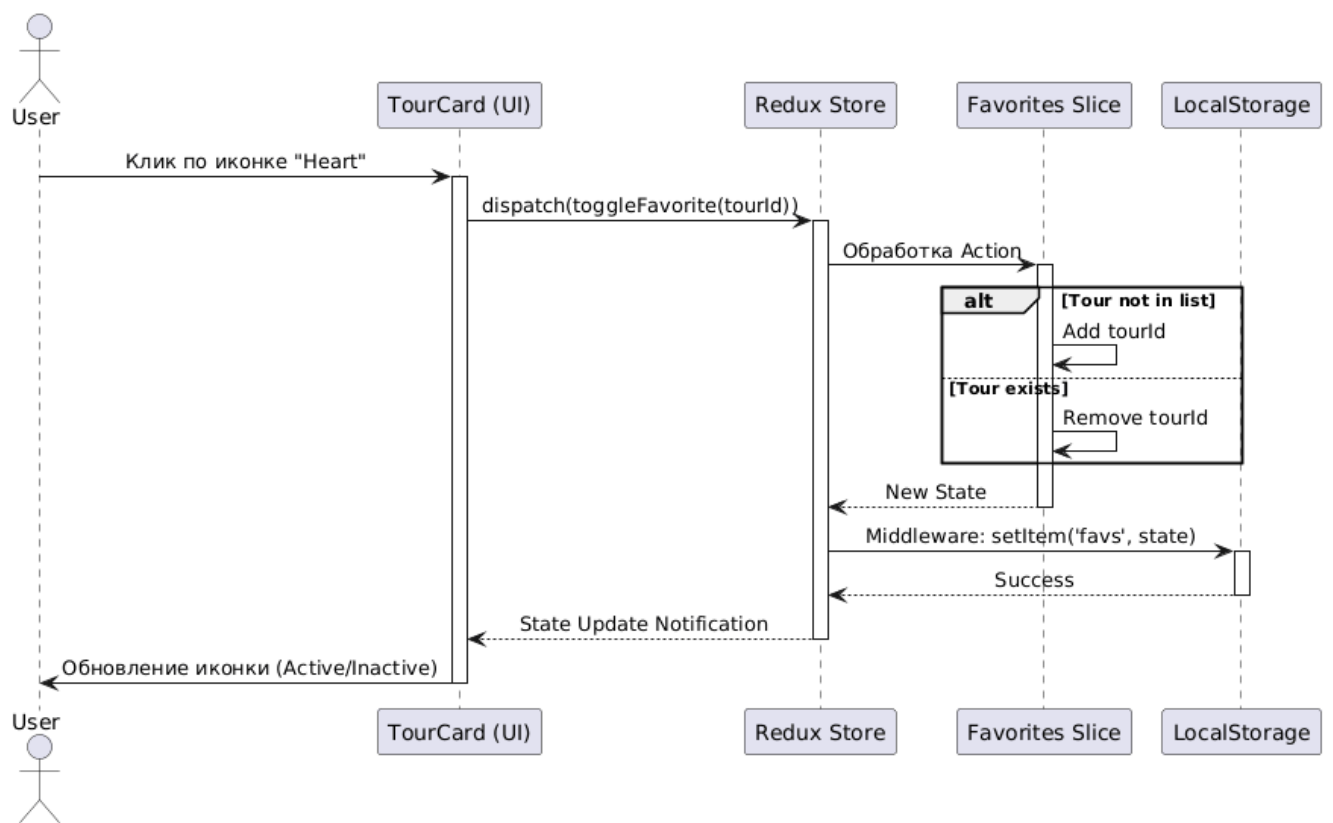


Рисунок 5.1 - Последовательность действий при добавлении в избранное



```

// store/slices/favoritesSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
  items: [], // Массив ID туров
};

export const favoritesSlice = createSlice({
  name: 'favorites',
  initialState,
  reducers: {
    toggleFavorite: (state, action) => {
      const tourId = action.payload;
      const index = state.items.indexOf(tourId);

      if (index >= 0) {
        // Если тур уже есть – удаляем
        state.items.splice(index, 1);
      } else {
        // Если нет – добавляем
        state.items.push(tourId);
      }
    },
    // Экшен для инициализации из LocalStorage
    hydrateFavorites: (state, action) => {
      state.items = action.payload;
    }
  },
});

export const { toggleFavorite, hydrateFavorites } = favoritesSlice.actions;
export default favoritesSlice.reducer;

```

Рисунок 5.2 - Фрагмент кода. Логика Redux Slice

Фактический срок реализации: 3 рабочих дня.

Комментарий принимающего лица:

«Функционал реализован в полном объеме. Гибридный подход с использованием Redux и LocalStorage обеспечивает отличный пользовательский опыт: данные не теряются, интерфейс работает плавно. Код написан чисто, логика выделена в отдельные сущности».

6. Написание модульных тестов для критических компонентов

Дата выдачи: 10 ноября 2025 г.

Автор задачи: руководитель практики.

Срок для исполнения: 4 рабочих дня.

Формулировка задачи:

С ростом кодовой базы увеличился риск возникновения регрессионных ошибок при внесении изменений. Была поставлена задача покрыть модульными тестами (Unit Tests) критически важные части приложения: утилиты форматирования цен, валидаторы форм и изолированные UI-компоненты (кнопки, инпуты).

Описание выполненных работ:

В качестве среды тестирования были настроены Jest и React Testing Library.

Разработаны тестовые сценарии (test cases) для утилитарных функций: проверена корректность форматирования валют, обработки дат и склонения существительных (например, «1 день», «5 дней»).

Написаны тесты для компонентов формы бронирования: смулированы события ввода данных (userEvent), проверено отображение сообщений об ошибках при некорректном заполнении полей и блокировка кнопки отправки.

Для тестирования компонентов, зависящих от Redux и Router, были созданы мок-обертки (mock wrappers), имитирующие окружение приложения.

```
// __tests__/BookingForm.test.js
import { render, screen, fireEvent } from '@testing-library/react';
import BookingForm from '../components/BookingForm';
import userEvent from '@testing-library/user-event';

describe('BookingForm Validation', () => {
  it('должен отображать ошибку при вводе некорректного email', async () => {
    // 1. Рендеринг компонента
    render(<BookingForm />);

    // 2. Получение ссылки на инпут
    const emailInput = screen.getByLabelText(/email/i);

    // 3. Эмуляция ввода пользователем невалидных данных
    await userEvent.type(emailInput, 'invalid-email-text');

    // 4. Снятие фокуса (blur) для триггера валидации
    fireEvent.blur(emailInput);

    // 5. Проверка наличия сообщения об ошибке
    const errorMessage = await screen.findByText(/введите корректный email/i);
    expect(errorMessage).toBeInTheDocument();
  });
});
```

Рисунок 6.1 - Фрагмент кода. Тест валидации email

Фактический срок реализации: 4 рабочих дня.

Комментарий принимающего лица:

«Внедрение тестирования — важный шаг к зрелости проекта. Тесты написаны грамотно, покрывают основные пограничные случаи (edge cases). Это позволит гарантировать стабильность базового функционала при дальнейшей разработке».

ЗАКЛЮЧЕНИЕ

Подводя итоги прохождения технологической практики, можно отметить, что поставленные цели были достигнуты в полном объёме. В ходе работы был получен практический опыт участия в разработке клиентской части веб-приложения, реализованного с использованием современного технологического стека, включающего React, Next.js, Redux и вспомогательные библиотеки для управления сессиями и авторизацией.

В процессе выполнения практических задач были освоены принципы построения архитектуры фронтенд-приложения, корректной инициализации глобального состояния, организации вложенности провайдеров и защиты пользовательских маршрутов. Реализация логики маршрутизации и валидации параметров позволила на практике понять важность обеспечения безопасности пользовательских сценариев и предотвращения некорректных состояний приложения. Работа с контекстом авторизации и cookie-хранилищами дала представление о реальных механизмах управления сессиями в веб-приложениях.

Особую ценность имел опыт работы с кодовой базой проекта, ориентированного на дальнейшее расширение и сопровождение. Это позволило сформировать понимание того, как отдельные технические решения влияют на масштабируемость, удобство поддержки и стабильность приложения в целом. Практика также способствовала развитию навыков анализа кода, самостоятельного принятия технических решений и соблюдения архитектурных соглашений проекта.

В результате прохождения практики были не только закреплены теоретические знания, полученные в университете, но и сформировано более осознанное понимание роли фронтенд-разработчика в процессе создания полноценного IT-продукта. Полученные навыки и опыт могут быть использованы как в дальнейшем обучении, так и при профессиональной деятельности в области веб-разработки и программной инженерии.